

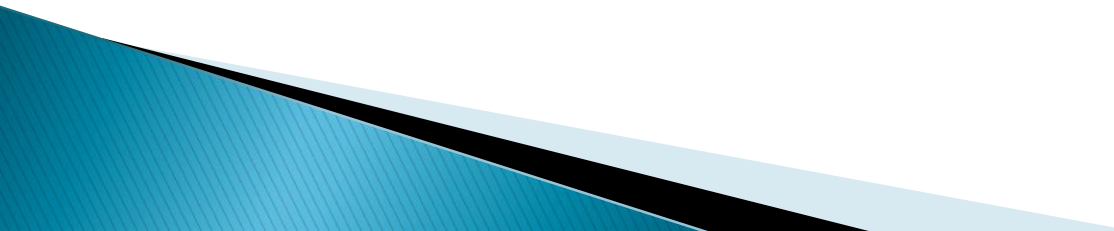
CSE 1101

Introduction to Programming

**Lecture 02:
Programming basics**



Content

- ▶ Structure of C Programming
 - ▶ Component in a program code
 - Preprocessor Directives / Library
 - Reserved Word
 - Data Type
 - Identifiers and their declarations
 - Comments
 - Escape Characters
 - Operators
 - Preprocessor Functions
 - ▶ Good Programming Style
- 

Sample of C Programming

```
#include <stdio.h>

const double TOWN_TAX_RATE = 0.0175 // constant declaration

int main(void) {

    /*Variable declarations*/

    double gross_income;
    double town_tax;

    /*Function body*/

    printf ("A PROGRAM THAT COMPUTES TOWN INCOME TAX\n");

    printf ("Enter gross income: RM ");
    scanf ("%lf", &gross_income);

    town_tax = TOWN_TAX_RATE * gross_income;

    printf ("Town tax is RM %f .", town_tax);

    return 0;

} /*End of function main*/
```

Preprocessor
Directives

Reserved Word

Data Type

Identifiers

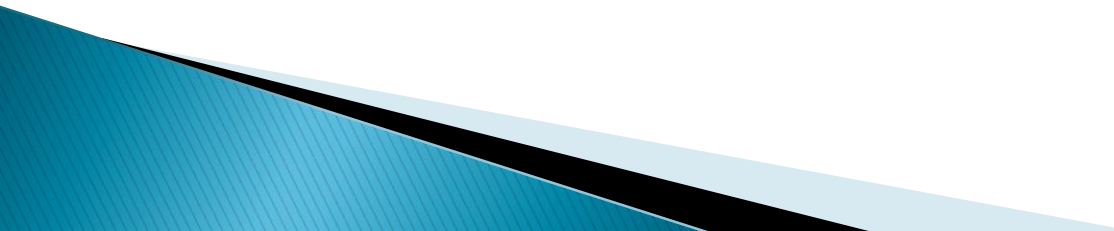
Comments

Escape
Character

Operators

Preprocessor
Functions /
statements

Preprocessor Directives / Library

- ▶ In C programming, the preprocessor directives start with **#include**
 - ▶ It is the specific line to tell the preprocessor to **include the contents of the predefined function** in to the program from the **header file**
 - ▶ The header file contains information and declarations used by the compiler when compiling the predefined functions
- 

Type of Header file and its functions

- ▶ Each header file consists of specific function to be applied

Header File	Explanation
<stdio.h>	Contains functions prototypes for the standard input output library function and the information to use them
<stdlib.h>	Contains function prototypes for conversions of numbers to text and text to numbers, memory allocation, random number, and other utility functions
<string.h>	Contains function prototypes for string processing functions
<time.h>	Contains function prototypes and types for manipulating the time and date
<math.h>	Contains function prototypes for math library functions

Reserved Words

- identify language entities, such as statements, data types, and language element attributes;
- They have special meaning to compiler, must appear in correct location in program, typed correctly, and used in correct context.
- Case sensitive: differentiates between lowercase & uppercase letter.
- **Example:** `const`, `double`, `int`, `return`

Sample of Reserved Word in C

asm	do	inline	short	typeid
auto	double	int	signed	typename
bool	dynamic_cast	long	sizeof	union
break	else	mutable	static	unsigned
case	enum	namespace	static_cast	using
catch	explicit	new	struct	virtual
char	extern	operator	switch	void
class	false	private	template	volatile
const	float	protected	this	wchar_t
const_cast	for	public	throw	while
continue	friend	register	true	
default	goto	reinterpret_cast	try	
delete	if	return	typedef	

Data Types

- ▶ A **data type** is a set of data values and a set of operations on those values.
- ▶ Usually has two classes of data types:
 - **Built-in data types**
 - **Programmer-defined data types** (covered in Structures)
- ▶ In addition, the C language has two classes of built-in data types:
 - **Fundamental data types:**
 - corresponds to the most common, fundamental storage units of a computer and the most common, fundamental ways of using such data.
 - **Example:** **int**, **char**, **double**, **float**, **void**
 - Some of these data types have **short**, **long**, **signed**, and **unsigned** variants.
 - **Derived data types:**
 - Derived from fundamental and programmer-defined data types using some operators.
 - Examples: **arrays**, **strings** and **structures**.

▶ Int

- is used to declare **numeric** program variables of integer type.

– Example:

- **int** counter;

Description:

- declares the program variable counter as an integer variable.
- We can store only **positive** or **negative** integers in the range -32768 until 32767.
- We can perform arithmetic & assignment operations on the variable counter

► Char

- is used to declare **character** variables.

- **Example:**

 - **char** myCharacter;

- **Description:**

 - declares the variable myCharacter to be type character.
 - Character variable stores any printable or nonprintable character in the computer's character set, including **lowercase, uppercase letters, decimal digits, special characters, and escape sequences.**
 - Such characters are represented in one byte (8 bits) of the computer's memory.

▶ double

- is used to declare a floating-point variables.

- Example:

 - `double gross_income;`

Description:

- declares the program variable `gross_income` as a double variable.
- We can store floating-point values in the computer's memory.

Data Type	Size (in Byte)
short	2
unsigned short	2
int	4
unsigned int	4
long	4
unsigned long	4

- ▶ **Unsigned** means the number is **always zero or positive, never negative**.
- ▶ **Signed** means the number **may be negative or positive** (or zero).
- ▶ If you don't specify signed or unsigned, the data type is presumed to be signed. Thus, signed short and short are the same.

Identifiers

- ▶ Also known as “variables”
- ▶ Words to represent and reference certain program entities
- ▶ Refer to a location in memory where value can be stored.
- ▶ Case sensitive: differentiates between lowercase & uppercase letter.
- ▶ **Example:** `student_id`, `name`, `Item15`, `number_of_strings`

► Rules for constructing Identifiers

- Identifiers can consist of the capital letters A to Z, the lowercase letters a to z, the digits 0 to 9, and the underscore character _.
- The first character must be a letter or an underscore.
- There is virtually no length limitation. However, in many implementations of the C language, the compilers recognize only the first characters as significant.
- There can be no embedded blanks.
- Reserved words cannot be used as identifiers.
- Identifiers are case sensitive. Therefore, result and Result, both valid examples of identifiers, are distinct.

- ▶ Examples of **legal** identifiers : `student_age`, `Item_25`, `counter`, `area_of_land`
- ▶ Examples of **illegal** identifiers :
 - `student age` → **embedded blank**
 - `continue` → **reserved word**
 - `15thFeb` → **the first character is a digit**
 - `Width+Length` → **special character +**

Tips to choose the name for identifier:

1. **Avoid excessively short and cryptic names such as `x` or `wt`. Instead, to add to the readability of your program, use reasonably descriptive names, such as `student_program` and `down_payment`.**
2. **Use underscore or capital letters to separate words in identifiers that consist of two or more words, such as `student_program` or `StudentProgram` are much easier to read than `studentprogram` .**

Comments

- ▶ **Program comments** : Are **explanations** or **annotations** that are included in a program for documentation and clarification purposes.
- ▶ It describe the purpose of a program, function or statement. They are completely ignored by the compiler during compilation, and they have no effect on program execution.
- ▶ A way of documentation within the coding
- ▶ To specify comment in C:
 - `/*This is a comments*/` or
 - `// This is another alternative of comment`

Data types in C

The types in C can be classified as follows:

S.N.	Types and Description
1	Basic Types: They are arithmetic types and consists of the two types: (a) integer types and (b) floating-point types.
2	Enumerated types: They are again arithmetic types and they are used to define variables that can only be assigned certain discrete integer values throughout the program.
3	The type void: The type specifier <i>void</i> indicates that no value is available.
4	Derived types: They include (a) Pointer types, (b) Array types, (c) Structure types, (d) Union types and (e) Function types.

Data types in C

Integer Types

Following table gives you details about standard integer types with its storage sizes and value ranges:

Type	Storage size	Value range
Char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
Int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
Short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
Long	4 bytes	-2,147,483,648 to 2,147,483,647
unsigned long	4 bytes	0 to 4,294,967,295

Data types in C

To get the exact size of a type or a variable on a particular platform, you can use the **sizeof** operator. The expressions **sizeof(type)** yields the storage size of the object or type in bytes. Following is an example to get the size of **int** type on any machine:

```
#include <stdio.h>
#include <limits.h>

int main()
{
    printf("Storage size for int : %d \n", sizeof(int));

    return 0;
}
```

Data types in C

Floating-Point Types

Following table gives you details about standard floating-point types with storage sizes and value ranges and their precision:

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
double	8 byte	2.3E-308 to 1.7E+308	15 decimal places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

The header file **float.h** defines macros that allow you to use these values and other details about the binary representation of real numbers in your programs. Following example will print storage space taken by a float type and its range values:

```
#include <stdio.h>
#include <float.h>

int main()
{
    printf("Storage size for float : %d \n", sizeof(float));
    printf("Minimum float positive value: %E\n", FLT_MIN );
    printf("Maximum float positive value: %E\n", FLT_MAX );
    printf("Precision value: %d\n", FLT_DIG );

    return 0;
}
```

Escape Characters

- ▶ Escape sequence indicates “special” character output

Escape Sequence	Name	Meaning of Escape Sequence
<code>\a</code>	Alert	Sounds a beep
<code>\b</code>	Backspace	Backs up one character
<code>\f</code>	Formfeed	Starts a new screen or page
<code>\n</code>	New line	Moves to beginning of next line
<code>\r</code>	Carriage return	Moves to beginning of current line
<code>\t</code>	Horizontal tab	
<code>\v</code>	Vertical tab	Moves to next tab position
<code>\\</code>	Backslash	Moves down a fixed amount
<code>\'</code>	Single quotation	Prints a backslash
<code>\"</code>	Double quotation	Prints a single quotation mark
<code>\?</code>	Double quotation	Prints a double quotation mark
<code>\000</code>	Question mark	Prints a quotation mark
<code>\XHHH</code>		Prints a character whose ASCII code is a one-to-three-digit octal value
		Prints a character whose ASCII code is a one-to-three-digit

Punctuation & Operators

- ▶ Punctuations serve as separators for functions or statements
- ▶ Examples:
 - `{}` , `()` , `;` , `:` , ...
- ▶ Operators can be
 - **Arithmetic operators**
 - Performing mathematical operation to the operands (A+B)
 - **Unary operators**
 - Act on single operand to produce a new value (e.g. -173)
 - **Relational and logical operators**
 - For the purpose of comparison (e.g. `>` , `<` , `>=` , `!=`);
 - **Assignment operators**
 - Assigning value to identifiers (e.g. `+=` , `++` , `--`)
 - **Conditioner operators**
 - Provide conditional statements `((i > 0) ? 0 : 1)`

Preprocessor Functions / statements

Statements

A **statement** is a specification of an action to be taken by the computer as the program executes.

```
/*Function body*/
```

```
printf ("A PROGRAM THAT COMPUTES TOWN INCOME TAX\n");
```

```
printf ("Enter gross income: RM ");
```

```
scanf ("%lf", &gross_income);
```

```
town_tax = TOWN_TAX_RATE * gross_income;
```

```
printf ("Town tax is RM %f .", town_tax);
```

```
return 0;
```

Output
statement

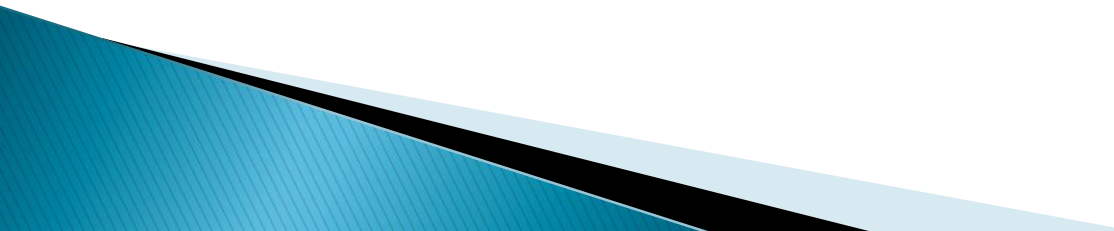
Input
statement

Compute &
stores the
result in
the
memory

Each such line is a statement.

Each statement causes the processor to do something.

Types of statements in programming

- ▶ There are several types of statements in :
 - 1.Expression statements
 - 2.Selection statements
 - 3.Repetition statements
 - 4.Jump statements
 - 5.Labeled statements
 - 6.Compound statements
- 

► Compound Statement

- Is a list of statements **enclosed in braces, { }.**
- Can contain any number of statements and declarations.
- Even though **all statements must end with semicolons, a compound statement does not need the semicolon delimiter after right brace, }.**

▶ Writing a Program

Programming Style

```
#include <stdio.h>

const double TOWN_TAX_RATE = 0.0175;
int main(void)
{
    double gross_income;
    double town_tax;

    printf ("A PROGRAM THAT \ COMPUTES TOWN INCOME TAX\n");
    printf ("Enter gross income: RM ");
    scanf ("%lf", &gross_income);

    town_tax = TOWN_TAX_RATE * gross_income;

    printf ("Town tax is RM %f .", town_tax);

    return 0;

}
```

Take a look and consider the coding above!!!!

Programming Style

What is Programming Style?

It is a collection of conventions, rules, and techniques that enable us to write programs in an elegant yet simple and efficient manner.

Techniques of style are highly personal and subjective and are mostly a matter of common sense.

In general, it is easier to understand and maintain such a program, even for someone other than the original developer.

We are going to discuss the following styles:

- Using Comments
- Variable Name
- Naming Style
- Indentation and Code Format
- Clarity
- Whitespace

Programming Style

Comments

Ideally, a comments serves two purposes:

1. Presents the computer with a set of instructions; and
2. Provides the programmer with a clear, easy-to-read description of what the programmer does.

A working but uncommented program is a time bomb waiting to explode.

Sooner or later someone will have to modify or upgrade the

```
/* ****  
* Program filename: myTesting.c *  
* Author: Md Shahid Uz Zaman *  
* Purpose: For Class Demonstration *  
* Usage: Run the program and message appear *  
* Date: 10th September 2012 *  
**** */
```

Programming Style

- Heading

The first comment should contain the name of the program. Also include a short description of what it does. You may have the most amazing program but it is useless if no one knows what it does.

- Author

You've gone to a lot of trouble to create this program. Take credit for it. Also, if someone else must later modify the program, he or she can come to you for information and help.

- Purpose

Why did you write this program? What does it do?

Programming Style

- Usage

In this section give a short explanation of how to run the program. In an ideal world, every program comes with a set of documents describing how to use it.

- References

Creative copying is a legitimate form of programming (if you don't break the copyright laws in the process). In the real world, it doesn't matter how you get a working program, as long as you get it; but, give credit where credit is due. In this section you should reference the original author of any work you copied.

- Restrictions

List any limits or restrictions that apply to the program, such as:
The data file must be correctly formatted; the program does not check for input errors.

- Revision history

This section contains a list indicating who modified the program and when and what changes have been made.

Programming Style

Variable Name

A variable is a place in the computer's memory for storing a value. Names can be any length and should be chosen so their meaning is clear.

The following declaration tells C that you are going to use three integer (int) variables named r, s, and t. **Avoid using abbreviations**

```
int r, s, t;
```

Consider another declaration:

```
int account_number;  
int balance_owed;
```

However, these examples still lacking of information. (i.e., is the balance_owed in ringgit or cents?). Therefore, it better if we added a comment after each declaration explaining what we are doing.

```
const double TOWN_TAX_RATE = 0.0175    // constant declaration
```


Programming Style

Indentation and Code Format

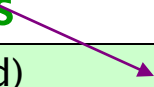
To make programs easier to understand, most programmers indent their programs.

The general rule for a C program is to indent one level for each new block or conditional.

There are two styles of indentation. Use any of the two, but make sure you are consistent in using the style that you choose.

Programming Style

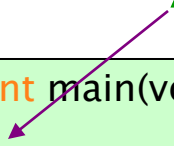
1st Style: braces are put on the same line as the statements



```
int main(void) {  
  
    /*Variable declarations*/  
  
    double gross_income;  
    double town_tax;  
  
    /*Function body*/  
  
    printf ("A PROGRAM THAT COMPUTES TOWN INCOME TAX\n");  
  
    printf ("Enter gross income: RM ");  
    scanf ("%lf", &gross_income);  
  
    town_tax = TOWN_TAX_RATE * gross_income;  
  
    printf ("Town tax is RM %f .", town_tax);  
  
    return 0;  
} /*End of function main*/
```

Programming Style

2nd Style: puts the curly braces on lines by themselves



```
int main(void)
{
    /*Variable declarations*/

    double gross_income;
    double town_tax;

    /*Function body*/

    printf ("A PROGRAM THAT COMPUTES TOWN INCOME TAX\n");

    printf ("Enter gross income: RM ");
    scanf ("%lf", &gross_income);

    town_tax = TOWN_TAX_RATE * gross_income;

    printf ("Town tax is RM %f .", town_tax);

    return 0;

} /*End of function main*/
```

```
/*Function body*/
```

```
printf ("A PROGRAM THAT COMPUTES TOWN INCOME TAX\n");
```

```
printf ("Enter gross income: RM ");
```

```
scanf ("%lf", &gross_income);
```

```
if (gross_income > 10000)
```

```
{
```

```
    printf("this person is reach!!");
```

```
    if (gross_income > 10000)
```

```
    {
```

```
        printf("this person is reach!!");
```

```
    }
```

```
}
```

```
town_tax = TOWN_TAX_RATE * gross_income;
```

```
printf ("Town tax is RM %f .", town_tax);
```

```
return 0;
```

```
/*End of function main*/
```

Programming Style

Clarity

A program should read like a technical paper.

It should be organized into sections and paragraphs. Procedures form a natural section boundary.

You should organize your code into paragraphs.

It is a good idea to begin a paragraph with a topic sentence comment and separate it from other paragraphs by a blank line.

```
// poor programming practice  
temp = box_x1;  
box_x1 = box_x2;  
box_x2 = temp; temp = box_y1;  
box_y1 = box_y2;  
box_y2 = temp;
```

Programming Style

A better version would be:

```
/* Purpose: Swap the two corners */
```

```
/* Swap X coordinate */
```

```
temp = box_x1;
```

```
box_x1 = box_x2;
```

```
box_x2 = temp;
```

```
/* Swap Y coordinate */
```

```
temp = box_y1;
```

```
box_y1 = box_y2;
```

```
box_y2 = temp;
```

Programming Style

Some styles conventions commonly used to produce readable code.

1. Insert blank lines between consecutive program sections, as shown below:

```
double gross_income;  
double town_tax;  
  
printf ("A PROGRAM THAT COMPUTES TOWN INCOME TAX\n");  
printf ("Enter gross income: RM ");  
scanf ("%lf", &gross_income); town_tax = TOWN_TAX_RATE * gross_income;
```

2. Make liberal use of clear and helpful comments.

What are the output?

Example 1:

```
#include <stdio.h>
```

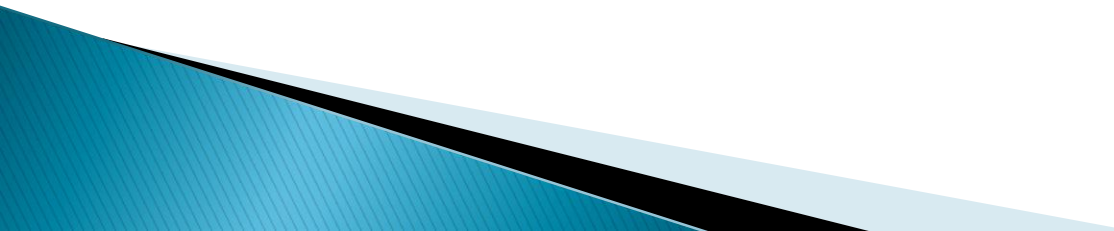
```
int main (void)
```

```
{
```

```
    printf("Welcome to C!\n");
```

```
    return 0;
```

```
}
```



Example 2:

```
#include <stdio.h>
```

```
int main (void)
```

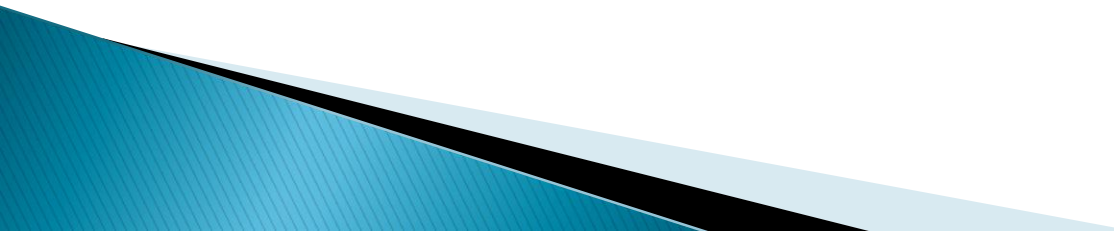
```
{
```

```
    printf("Welcome");
```

```
    printf("to C!\n");
```

```
    return 0;
```

```
}
```



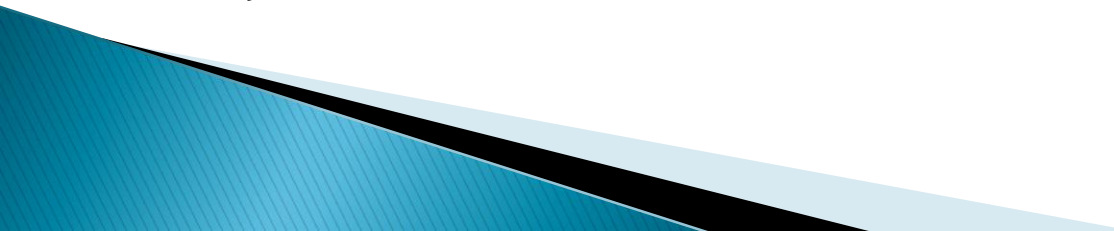
Example 3:

```
#include <stdio.h>
```

```
int main(void)  
{
```

```
    printf("Welcome \n to\n C!\n");  
    return 0;
```

```
}
```



Any Questions??

Exercises

1) Print the hello world sentence.

Output:

Hello World!

2) Write simple c program to add two integers.

Output:

Enter first integer

40

Enter second integer

70

Sum is 110



QUIZ

1. Which of the following statements should be used to obtain a remainder after dividing 3.14 by 2.1 ?

A `rem = 3.14 % 2.1;`

▪

B `rem = modf(3.14, 2.1);`

▪

C `rem = fmod(3.14, 2.1);`

▪

D Remainder cannot be obtain in floating point division.

▪

Answer: Option C

Explanation:

`fmod(x, y)` - Calculates x modulo y, the remainder of x/y.
This function is the same as the modulus operator.
But `fmod()` performs floating point divisions.

QUIZ

2 Which of the following special symbol allowed in a variable . name?

A * (asterisk)

▪

C - (hyphen)

▪

B | (pipeline)

▪

D _ (underscore)

▪

Answer: Option **D**

Explanation:

Variable names in C are made up of letters (upper and lower case) and digits. The underscore character ("_") is also permitted. Names must not begin with a digit.

QUIZ

3 How would you round off a value from 1.66 to 2.0?

▪

A `ceil(1.66)`

B `floor(1.66)`

▪

C `roundup(1.66)`

▪

D `roundto(1.66)`

▪

▪

Answer: Option **A**

Explanation: `ceil()` function truncates a number to its upper integer.

QUIZ

4 By default a real number is treated as a

- - A** float
 - B** double
 - C** long double
 - D** far double

Answer: Option **B**

Explanation:

When the accuracy of the floating point number is insufficient, we can use the `double` to define the number. The `double` is same as `float` but with longer precision and takes double space (8 bytes) than `float`. To extend the precision further we can use `long double` which occupies 10 bytes of memory space.

QUIZ

5. Which of the following is a correct comment?

- A. `*/ Comments */`
- B. `** Comment **`
- C. `/* Comment */`
- D. `{ Comment }`

Answer: Option **C**

QUIZ

6. Which of the following is not a correct variable type?

- A. float
- B. real
- C. int
- D. double

Answer: Option B

QUIZ

7. Which of the following is the correct operator to compare two variables?

A. :=

B. =

C. equal

D. ==

Answer: Option D

QUIZ

8. A number such as 45.567 needs to be stored in a variable of which data type?

- A. Int
- B. Char
- C. Double
- D. Real

Answer: Option C

QUIZ

10. The "\n" character does which of the following operations?

- A. Double line spacing
- B. Character deletion
- C. Character backspace
- D. Places cursor on the next line

Answer: Option D